



Claude Code Skills

Moving specialized instructions out of CLAUDE.md and into on-demand, progressively-loaded capabilities — reference guide

🔗 WHAT'S INSIDE

1. Why Skills — the CLAUDE.md bloat problem
2. What a Skill is — progressive disclosure, three levels
3. Anatomy of a SKILL.md file
4. Where Skills live & how they're invoked
5. Worked example — the automation-standards Skill
6. Splitting a command's logic into a Skill
7. Skills vs Slash Commands vs CLAUDE.md
8. Glossary & self-test

Why Skills — the CLAUDE.md Bloat Problem

Everything in CLAUDE.md loads on every message, whether it's relevant or not

CLAUDE.md is loaded into context the moment a session starts, and stays there for every message sent afterward. That's exactly what makes it reliable for standing project rules — and exactly what makes it expensive to overload.

📁 The pattern, in two real repos

Test Case Generator

CLAUDE.md holds the full test-case format, field-by-field guidance, coverage requirements, and generation rules. Useful when generating test cases — dead weight for every other request.

Playwright automation repo

Ten automation standards live in CLAUDE.md — selectors, waits, test isolation, Page Objects, data and config rules. All ten load on every message, from fixing a config file to writing a page object.

⚠ THE ACTUAL QUESTION

Most requests in a session aren't "generate test cases" or "review this spec against our standards." They're everyday work — update a config, explain a function, write the next slash command. Do those tasks need the entire rulebook loaded every single time? Not necessarily — and as a project grows, that rulebook only gets bigger.

📖 The shelf analogy

CLAUDE.md

A page that's always open on the desk — every word read, every single message

A Skill

A book on the shelf — only the title is visible until you need to actually open and read it

➡ PROGRESSIVE DISCLOSURE

Keep general information available at all times, and load more detailed information only when it becomes relevant. This is the organizing idea behind Skills — and it's Anthropic's own term for the mechanism, not just a teaching analogy.

What a Skill Actually Is

A directory on disk, read in stages — not a prompt fragment kept in memory

A Skill is a folder containing a `SKILL.md` file, optionally alongside supporting files. Claude Code discovers Skills automatically and reads them from the filesystem using the same tools it uses to read any other file — nothing is preloaded wholesale into every conversation.

Three loading levels

LEVEL	WHEN LOADED	TOKEN COST	CONTENT
1 • Metadata	Always, at startup	~100 tokens per Skill	<code>name</code> and <code>description</code> from the frontmatter — this is what Claude matches your request against
2 • Instructions	Only when the Skill triggers	Under ~5k tokens	The body of <code>SKILL.md</code> — the actual workflow, rules, or checklist
3 • Resources	Only as referenced	None until accessed	Bundled files — reference docs, scripts, templates — read or run only when the instructions point to them

i WHAT THIS MEANS IN PRACTICE

Until a Skill is triggered, only its name and one-line description sit in context — roughly the cost of a single sentence. Installing many Skills carries almost no penalty; the real cost only shows up once a Skill is actually invoked and its body loads.

How triggering works



Automatic

Claude reads the task, matches it against every Skill's description, and loads the one that fits — no explicit command needed



Explicit

Type `/skill-name` directly, the same way a slash command is invoked

Because matching happens against the `description` field specifically, a vague description ("test-case rules") gives Claude far less to match against than a specific one ("the test-case format and coverage rules for this project — use when generating test cases from a user story").

➡ PERSISTENCE WITHIN A SESSION

Once a Skill's body loads into a conversation, it stays there for the rest of that session — Claude doesn't re-read the file on every later turn. The cost is paid once per invocation, not once per message.

Anatomy of a SKILL.md File

Frontmatter that controls discovery and behavior, plus the instruction body

```
---
name: tc-generator
description: The test-case format and coverage rules for this project.
  Use when generating test cases from a user story.
---

# Test case generation
[the format, field guidance, and coverage rules go here]
```

Key frontmatter fields

FIELD	REQUIRED?	WHAT IT DOES
name	No*	Display label. In Claude Code, the folder name is what actually becomes the <code>/command-name</code> — this field is optional and only sets the label shown in listings
description	Recommended	What the Skill does and when to use it. This is the only content Claude sees before deciding whether to load the full Skill — the single most important field
allowed-tools	No	Pre-approves specific tools so Claude doesn't have to ask permission during the turn that invokes the Skill — see the correction below
disable-model-invocation	No	Set <code>true</code> so only a human can trigger it with <code>/name</code> — Claude won't invoke it automatically. Useful for anything with side effects, like a deploy
user-invocable	No	Set <code>false</code> so only Claude can trigger it — hides it from the <code>/</code> menu. Useful for background knowledge that isn't a meaningful user action

⚠ CORRECTION: WHAT ALLOWED-TOOLS ACTUALLY DOES

`allowed-tools: Read` does **not** lock Claude into a read-only sandbox where editing or running commands is impossible. It pre-approves the `Read` tool so Claude can use it without a permission prompt during that turn — every other tool is still technically callable, but falls back to your normal permission settings (which may still ask, allow, or deny). Listing a tool grants a shortcut past the prompt; it doesn't restrict the tool set to only what's listed.

🔒 *name* required outside Claude Code

Claude Code itself treats every frontmatter field as optional except `description`. But the open Agent Skills standard — used when uploading a Skill to claude.ai or the API — requires both `name` and `description`, with stricter rules: `name` must be lowercase letters, numbers and hyphens only, max 64 characters, and can't contain the words "anthropic" or "claude". Worth knowing if a Skill built for one repo is ever shared more widely.

Where Skills Live & How They're Invoked

Personal, project, or shared — and the recent merge with slash commands

```
~/.claude/skills/<skill-name>/SKILL.md    ← PERSONAL: all your projects on this machine
.claude/skills/<skill-name>/SKILL.md      ← PROJECT: this repo only, commit to share with the team
<subdir>/.claude/skills/<name>/SKILL.md   ← NESTED: loads for work under that subdirectory
```

🔴 IMPORTANT UPDATE: COMMANDS AND SKILLS HAVE MERGED

Anthropic's current documentation states plainly: "*Custom commands have been merged into skills.*" A file at `.claude/commands/deploy.md` and a Skill at `.claude/skills/deploy/SKILL.md` both create `/deploy` and work the same way. Existing `.claude/commands/` files keep working — this is the older, simpler format. A Skill folder just adds optional extras: a directory for supporting files, frontmatter to control who can invoke it, and automatic loading when Claude judges it relevant. For new work, a Skill is the recommended shape.

🔗 Reference vs. task content

Anthropic's guidance distinguishes two shapes of Skill content, and it's worth deciding which one a given Skill is before writing it:

📖 Reference content

Knowledge Claude applies alongside your request — conventions, patterns, standards. This is what `automation-standards` is: Claude reads it and applies it while reviewing or writing code, without necessarily "running" anything.

⚡ Task content

Step-by-step instructions for a specific action — a deploy, a commit, a multi-step review process. Often paired with `disable-model-invocation: true` so it only runs when explicitly typed as `/name`.

🔑 KEEP THE BODY CONCISE

Anthropic's own guidance: keep SKILL.md under roughly 500 lines. Once a Skill loads, its content stays in context for the rest of the session — every line is a recurring cost, not a one-time read. State what to do rather than narrating why; move detailed reference material into supporting files instead.

Building automation-standards

Moving ten Playwright standards out of CLAUDE.md and into a Skill

📄 .claude/skills/automation-standards/SKILL.md

```
---
name: automation-standards
description: Playwright automation standards for this repo — selectors, waits,
  test isolation, Page Objects, data and config. Use when writing or
  reviewing specs or page objects.
allowed-tools: Read
---

# Automation standards

Each standard: the rule, why it matters, and a bad → good example.
[$1-$10 content here]
```

The ten standards that used to live directly in CLAUDE.md — selectors, waits, test isolation, Page Objects, data and config — move into this file, word for word. CLAUDE.md no longer carries them at all.

🔗 What CLAUDE.md looks like afterward

```
# Code review

For anything about spec or page-object quality, use the automation-standards
skill.
```

🔗 What stays in CLAUDE.md — and why

SECTION	WHY IT DOESN'T MOVE TO A SKILL
What this repo is	Every session starts from zero — no memory of the last one. This orients Claude in one shot: what the repo is, what's off-limits, the one thing to never do. It has to be there before Claude does anything.
Commands	<code>npm test</code> isn't tied to one task — it's reached for constantly, across every kind of work. Hiding it behind a trigger Claude has to go looking for defeats the point; it should just always be there.
Architecture	"Specs use fixtures," "Page Objects hold the selectors" — true for any change in this repo, whether fixing a bug or writing a new spec. Not a sometimes-fact.

👉 THE ONE RULE TO HOLD ONTO

Skills are for things you only need *sometimes*. CLAUDE.md is for things you need *basically always*. Ask "do I need this on almost every task, or only some of them?" — the answer tells you where it belongs.

Two loose ends worth catching in a real migration: an opening paragraph that says "...against the automation standards defined in CLAUDE.md" should become "...against the `automation-standards` skill," and any step heading like "Compare against CLAUDE.md" needs the same update — the standards don't live there anymore.

When a Command's Process Goes Missing

A Skill can hold standards — but what about a multi-step process?

A real problem shows up once `/review` exists as a command and `automation-standards` exists as a Skill: the actual step-by-step review process — identify files → check structure → check locators → check Page Objects → check Playwright practices → compare against standards → give a verdict — only lives inside `review.md`, a command file. Commands only run when someone explicitly types `/review`.

⚠ THE GAP THIS CREATES

If someone just asks in plain English, "can you review this test file," Claude might reach for the `automation-standards` Skill on its own — the description matches. But it never sees the 7-step process or the expected verdict format, because that detail sits in a file Claude only reads when `/review` is typed explicitly.

🔧 The fix: three files, one job each

```
.claude/skills/automation-standards/
├─ SKILL.md           ← standards (S1-S10), plus one new line:
|                       "for the review steps, see review-process.md"
└─ review-process.md  ← the 7-step process + verdict format, pasted in
                        unchanged, except "CLAUDE.md" → "the automation-standards skill"

.claude/commands/review.md ← stays in place, /review keeps working — but now just says
                            "use the automation-standards skill" and nothing else
```

FILE	ROLE AFTER THE SPLIT
SKILL.md	Holds the standards as before, plus one pointer line to the new file for the actual review steps
review-process.md	A new supporting file next to SKILL.md, inside the same Skill folder — the pasted 7-step process and verdict list, with every "CLAUDE.md" reference changed to "the automation-standards skill"
.claude/commands/review.md	Unchanged location, so <code>/review</code> keeps working as a slash command — but now very short: all the real detail has moved into the Skill

🔑 WHY THIS WORKS EITHER WAY IN

Now the process is reachable two ways: explicitly via `/review` (which points at the Skill), and automatically when Claude judges a plain-English request matches the Skill's description — because the process itself is a supporting file *inside* the Skill, not trapped in a command file that only fires on an exact trigger.

Skills vs Commands vs CLAUDE.md, & Self-Test

Three mechanisms, three different jobs — and how to tell them apart

	CLAUDE.MD	SLASH COMMAND	SKILL
When loaded	Always, at session start	Only when explicitly typed	Description always loaded; body only when triggered
Who can trigger	N/A — always on	You (and Claude, unless restricted)	You <i>or</i> Claude automatically, based on the description
Best for	Standing facts true on almost every task	An explicit, on-demand action with a clear trigger	Specialized knowledge or a process only relevant sometimes
Can bundle files?	Via <code>@imports</code> (still fully loaded)	No — single file	Yes — scripts, references, templates, loaded only as needed

i THEY COMPOSE, NOT COMPETE

In practice these three work together: CLAUDE.md tells Claude what this repo is and points to the Skill; the Skill holds the specialized rules or process; the command gives that Skill a short, predictable, human-triggerable name. Removing content from CLAUDE.md into a Skill doesn't remove the command — it just makes the command thin.

Glossary

TERM	MEANING
Progressive disclosure	Loading general information always, and detailed information only when relevant
SKILL.md	The required file inside a Skill folder — frontmatter plus instructions
allowed-tools	Frontmatter field that pre-approves tools without a permission prompt, for the turn that invokes the Skill
disable-model-invocation	Frontmatter field restricting a Skill to manual, human-typed invocation only
Supporting file	A file inside a Skill folder besides SKILL.md — read only when the instructions reference it

✓ Self-test

Q1

Why does putting all ten automation standards in CLAUDE.md waste context on most requests?

Q2

What are the three loading levels, and when does each one enter context?

Q3

Does `allowed-tools: Read` stop Claude from ever editing a file? Why or why not?

Q4

Why did the review process need to move out of review.md and into the Skill folder?

Q5

Give the one-line test for whether something belongs in CLAUDE.md or a Skill.